Computer Science & IT

DPP + Test + PYQ

+ Notes



Data Structure & Programming



Stack

Lecture No. 04

By- Abhishek Sir

Recap of Previous Lecture



- Topic — Expression
- Topic — Infix, prefix, postfix
- Topic — precedence & Associativity
- Topic — using stack
- Topic

Slide

Topics to be Covered



- Topic
- Topic
- Topic
- Topic
- Topic

Reversal Method Infix → prefix

How to evaluate ( ) using stack

Evaluation of postfix & prefix expression

Slide



Topic : Question

Let s be a stack of size $n \geq 1$. Starting with the empty stack, suppose we push the first n natural numbers in sequence, and then perform n pop operations. Assume that *Push* and *Pop* operations take X seconds each, and Y seconds elapse between the end of one such stack operation and the start of the next operation. For $m \geq 1$, define the stack-life of m as the time elapsed from the end of Push (m) to the start of the pop operation that removes m form S. The average stack life of an element of this stack is

(A) $n(X + Y)$

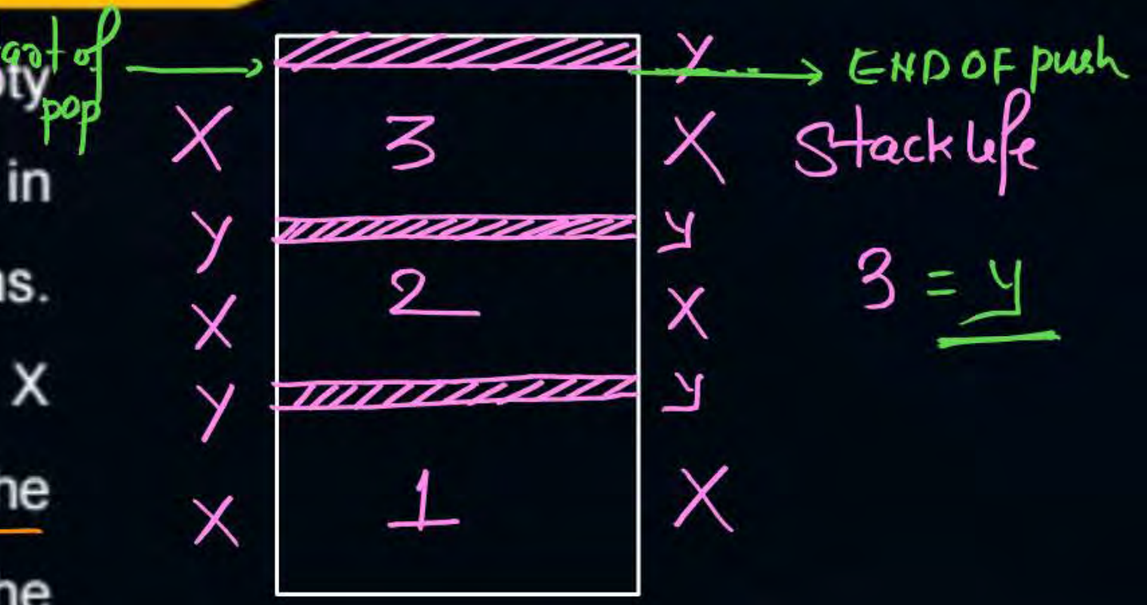
(B) $3Y + 2X$

(C) $n(X + Y) - X$

(D) $Y + 2X$

Topic : Question

Let s be a stack of size n ≥ 1. Starting with the empty stack, suppose we push the first n natural numbers in sequence, and then perform n pop operations. Assume that *Push* and *Pop* operations take X seconds each, and Y seconds elapse between the end of one such stack operation and the start of the next operation. For m ≥ 1, define the stack-life of m as the time elapsed from the end of Push (m) to the start of the pop operation that removes m form S. The average stack life of an element of this stack is



push pop X seconds

Y

push || push

Y

push || pop

Y

pop || push

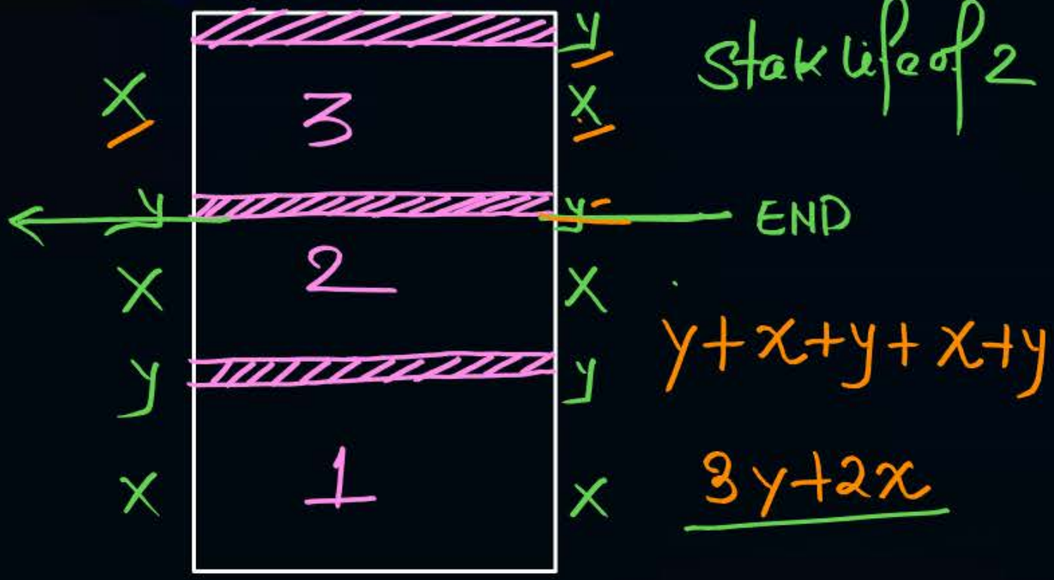
Y

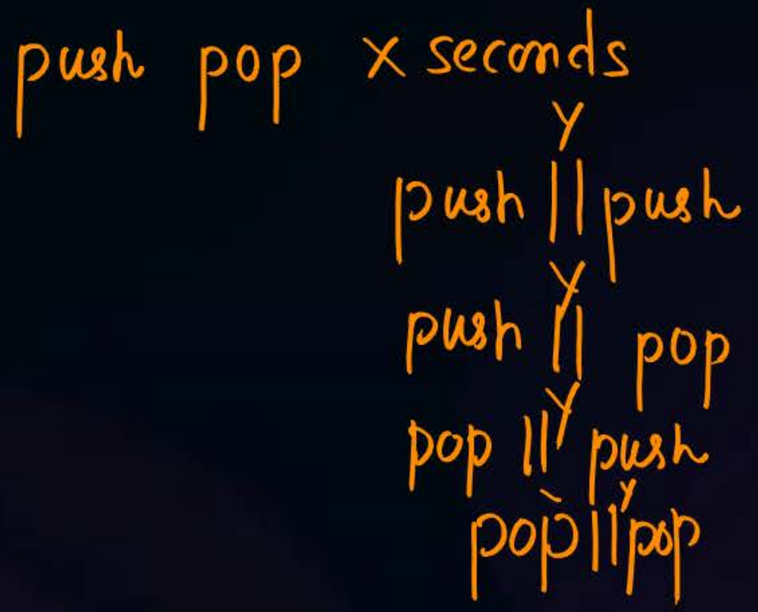
pop || pop



Topic : Question

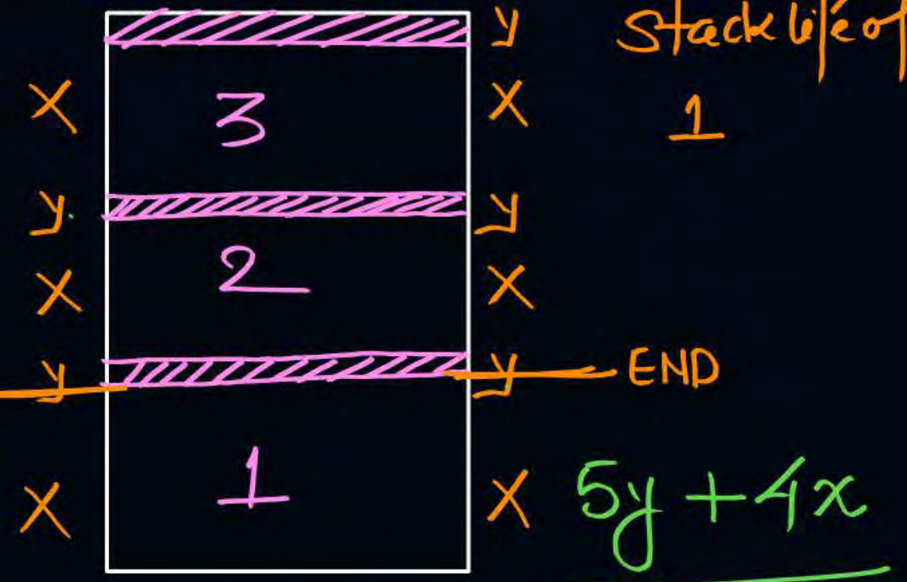
Let s be a stack of size n ≥ 1. Starting with the empty stack, suppose we push the first n natural numbers in sequence, and then perform n pop operations. Assume that *Push* and *Pop* operations take X seconds each, and Y seconds elapse between the end of one such stack operation and the start of the next operation. For m ≥ 1, define the stack-life of m as the time elapsed from the end of Push (m) to the start of the pop operation that removes m form S. The average stack life of an element of this stack is





Topic : Question

Let s be a stack of size n ≥ 1. Starting with the empty stack, suppose we push the first n natural numbers in sequence, and then perform n pop operations. Assume that *Push* and *Pop* operations take X seconds each, and Y seconds elapse between the end of one such stack operation and the start of the next operation. For m ≥ 1, define the stack-life of m as the time elapsed from the end of Push (m) to the start of the pop operation that removes m form S. The average stack life of an element of this stack is



push pop x seconds

y

push || push

y

push || pop

y

pop || push

y

pop || pop

n top — y

top-1 — 2x+3y

top-2 — 4x+5y

⋮

Bottom

1 — 2(n-1)x + (2n-1)y

$2x+4x+6x+\cdots 2(n-1)x$

$$Avg = \frac{+y+3y+5y+\cdots+(2n-1)y}{n}$$



Topic : Question

Let s be a stack of size n ≥ 1. Starting with the empty stack, suppose we push the first n natural numbers in sequence, and then perform n pop operations. Assume that *Push* and *Pop* operations take X seconds each, and Y seconds elapse between the end of one such stack operation and the start of the next operation. For m ≥ 1, define the stack-life of m as the time elapsed from the end of Push (m) to the start of the pop operation that removes m form S. The average stack life of an element of this stack is

$3 \rightarrow y$

$2 \rightarrow 2x+3y$

$1 \rightarrow 4x+5y$

$$\text{Avg} = \frac{y+2x+3y+4x+5y}{3}$$

$$= \frac{6x+9y}{3} = 2x+3y$$



Topic : Question

Let s be a stack of size n ≥ 1. Starting with the empty stack, suppose we push the first n natural numbers in sequence, and then perform n pop operations. Assume that *Push* and *Pop* operations take X seconds each, and Y seconds elapse between the end of one such stack operation and the start of the next operation. For m ≥ 1, define the stack-life of m as the time elapsed from the end of Push (m) to the start of the pop operation that removes m form S. The average stack life of an element of this stack is



Topic : Question

Let s be a stack of size n ≥ 1. Starting with the empty stack, suppose we push the first n natural numbers in sequence, and then perform n pop operations. Assume that *Push* and *Pop* operations take X seconds each, and Y seconds elapse between the end of one such stack operation and the start of the next operation. For m ≥ 1, define the stack-life of m as the time elapsed from the end of Push (m) to the start of the pop operation that removes m form S. The average stack life of an element of this stack is



Infix to Postfix: a↑b*c/(d*e–f)

Symbol	→ Stack					Output
a						a



Infix to Postfix: a↑b*c/(d*e–f)

Infix to Postfix: a↑b*c/(d*e–f)

Symbol	Stack					Output
a						a
↑	↑					a



Infix to Postfix: a↑b*c/(d*e–f)

Infix to Postfix: a↑b*c/(d*e–f)

Symbol	Stack					Output
a						a
↑	↑					a
b	↑					ab

Infix to Postfix: a↑b*c/(d*e−f)

Symbol	Stack					Output
a						a
↑	↑					a
b	↑					ab
*	↑					* Is having lower precedence then pop ↑ and print , push *

LPOPU-S



Infix to Postfix: a↑b*c/(d*e–f)

Infix to Postfix: a↑b*c/(d*e–f)

Symbol	Stack					Output
a						a
↑	↑					a
b	↑					ab
*	↑					* Is having lower precedence then pop ↑ and print , push *
	*					ab ↑



Infix to Postfix: a↑b*c/(d*e–f)

Infix to Postfix: a↑b*c/(d*e–f)

Symbol	Stack					Output
a						a
↑	↑					a
b	↑					ab
*	↑					* Is having lower precedence then pop ↑ and print , push *
	*					ab ↑
c	*					ab↑c

Infix to Postfix: a↑b*c/(d*e–f)

Infix to Postfix: a↑b*c/(d*e–f)

Symbol	Stack					Output
a						a
↑	↑					a
b	↑					ab
*	↑					* Is having lower precedence then pop ↑ and print , push *
	*					ab ↑
c	*					ab↑c
/	*					/ Is having lower precedence then pop * and print , push /(Left associativity)



Infix to Postfix: a↑b*c/(d*e–f)

Infix to Postfix: a↑b*c/(d*e–f)

Symbol	Stack					Output
a						a
↑	↑					a
b	↑					ab
*	↑					* Is having lower precedence then pop ↑ and print , push *
	*					ab ↑
c	*					ab↑c
/	*					/ Is having lower precedence then pop * and print , push /(Left associativity)
	/					ab↑c*

Infix to Postfix: a↑b*c/(d*e–f)

separate

Symbol	Stack					Output
a						a
↑	↑					a
b	↑					ab
*	↑					* Is having lower precedence then pop ↑ and print , push *
	*					ab ↑
c	*					ab↑c
/	*					/ Is having lower precedence then pop * and print , push /(Left associativity)
	/					ab↑c*
(	/	(				ab↑c*



Infix to Postfix: a↑b*c/(d*e–f)

Infix to Postfix: a↑b*c/(d*e–f)

Symbol	Stack					Output
d	/	(				ab↑c*d



Infix to Postfix: a↑b*c/(d*e–f)

Infix to Postfix: a↑b*c/(d*e–f)

Symbol	Stack					Output
d	/	(				ab↑c*d
*	/	(	*			ab↑c*d



Infix to Postfix: a↑b*c/(d*e–f)

Infix to Postfix: a↑b*c/(d*e–f)

Symbol	Stack					Output
d	/	(				ab↑c*d
*	/	(	*			ab↑c*d
e	/	(	*			ab↑c*de



Infix to Postfix: a↑b*c/(d*e–f)

Infix to Postfix: a↑b*c/(d*e–f)

Symbol	Stack					Output
d	/	(				ab↑c*d
*	/	(	*			ab↑c*d
e	/	(	*			ab↑c*de
-	/	(	*			- Is having lower precedence then pop * and print , push -



Infix to Postfix: a↑b*c/(d*e–f)

Infix to Postfix: a↑b*c/(d*e–f)

Symbol	Stack					Output
d	/	(				ab↑c*d
*	/	(	*			ab↑c*d
e	/	(	*			ab↑c*de
-	/	(	*			- Is having lower precedence then pop * and print , push -
-	/	(	-			ab↑c*de*



Infix to Postfix: a↑b*c/(d*e–f)

Infix to Postfix: a↑b*c/(d*e–f)

Symbol	Stack					Output
d	/	(				ab↑c*d
*	/	(	*			ab↑c*d
e	/	(	*			ab↑c*de
-	/	(	*			- Is having lower precedence then pop * and print , push -
-	/	(	-			ab↑c*de*
f	/	(	-			ab↑c*de*f



Infix to Postfix: a↑b*c/(d*e–f)

Infix to Postfix: a↑b*c/(d*e–f)

Symbol	Stack					Output
d	/	(				ab↑c*d
*	/	(	*			ab↑c*d
e	/	(	*			ab↑c*de
-	/	(	*			- Is having lower precedence then pop * and print , push -
-	/	(	-			ab↑c*de*
f	/	(	-			ab↑c*de*f
)	/					Pop till ~~closing~~ opening bracket and print pop opening bracket



Infix to Postfix: a↑b*c/(d*e–f)

Infix to Postfix: a↑b*c/(d*e–f)

Symbol	Stack					Output
d	/	(				ab↑c*d
*	/	(	*			ab↑c*d
e	/	(	*			ab↑c*de
-	/	(	*			- Is having lower precedence then pop * and print , push -
-	/	(	-			ab↑c*de*
f	/	(	-			ab↑c*de*f
)	/					Pop till closing bracket and print
	/					ab↑c*de*f-

Infix to Postfix: **a↑b*c/(d*e–f)**

Symbol	Stack					Output
d	/	(				ab↑c*d
*	/	(	*			ab↑c*d
e	/	(	*			ab↑c*de
-	/	(	*			- Is having lower precedence then pop * and print , push -
-	/	(	-			ab↑c*de*
f	/	(	-			ab↑c*de*f
)	/					Pop till closing bracket and print
	/					ab↑c*de*f-
						ab↑c*de*f-/

1. If bracket encounter in expression
   push the opening bracket

2. Treat expression in bracket as independent expression &
   perform same set of operation as for expression without bracket
   HPU / LPOPU-S ('(' is the marker)

3. If input is closing bracket ')' the pop every operator & print
   pop opening bracket & discard it.

Infix to Prefix (Reverse Approach)

$a + \underset{2}{(b} * \underset{1}{c)} \Rightarrow a + * bc \Rightarrow + a * bc$

1. Reverse the expression $\quad c * b + a$

2. follow same Method to convert to postfix

$cb * a +$

3. Reverse the expression

$\boxed{+ a * bc}$

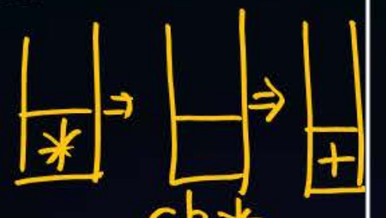
Input : c
output : c

Input * 

output : c

Input : b 

output : cb

Input : + 

output cb

Input : a 

output cb*a

END OF INPUT
output cb*a+

Infix to Prefix (Reverse Approach)

(1) $a+_1 b -_2 c$ → $+ab-c$ → $-+abc$

original Left Associative precedence of original expression

Reverse - $c-b+a$

Convert into postfix : $cba+-$

Reverse : $-+abc$

Operator stack

Input : c
output : c

Input : −
output : c



Input : b
output cb



Input : +
output : cb



Input : a
output : cba

END OF INPUT
cba + −

: $a+(b-c) \Rightarrow abc-$

$\Rightarrow$

Bracket get the first precedence



Topic : Evaluation of Postfix Expression

```
opndstk - the eapty stack;
    /* scan the input string reading one element at a time into symb */
    Whi1e (not end of input) {
            symb = next input character;
            if (syab is an operand)
                    push(opndstk, symb);
            else {
            /* symb is an operator */
                    Opnd2 = pop(opndstk);
                    opndl = pop(opndstk);
                    value = result of applying symb to opndl and opnd2;
                    push(opndstk,value);
            } /* end else */
    } /*end while */
    return(pop(opndstk));
```

6 6 2 2 3 3 + + − 3 8 2 / + * 2 ↑ 3 +

		3							
	2	2	5						
6	6	6	6						

$Opnd_2 = 3$ value $= 2+3 = 5$

$Opnd_1 = 2$

Topic : Evaluation of Postfix Expression

Single scan can evaluate the entire expression

6 2 3 + − 3 8 2 / + * 2 ↑ 3 +

6	2	−	+	−	3	8	2	/	+	*	2	↑	3	+
							2							
		3				8	8	8/2 =4						
	2	2	2+3 =5		3	3	3	3	3+4 =7		2		3	
6	6	6	6	6-5 =1	1	1	1	1	1	1*7 =7	7	7↑2 =49	49	49+3 =52

↑ Stack Operand stack

a + b

Operand$_1$ op Operand$_2$

top = operand$_2$

top-1 = operand$_1$

Operand$_1$ Operand$_2$ op

a b +

Slide



Topic : Evaluation of Postfix Expression

23 * 422 * / + 35 + -

Correct Ans is - 1

* * / + + −

		2														
		2	4			5										
3		4	4	1		3	8									
2	6	6	6	6	7	7	7	7-8 = -1								

Slide



Topic : Question

#Q. The following postfix expression with single digit operands is evaluated using a stack:

823^-/ 23 * + 51 * -

Note that ^ is the exponentiation operator. The top two elements of the stack after the first * is evaluated are:

(A) 6, 1

(B) 5, 7

(C) 3, 2

(D) 1, 5

↑		/		*	
3			3		
2	2^3 = 8		2	6	
8	8	8/8 = 1	1	1	

first multiplication

Slide

Topic : Question

Consider two binary operators '$\uparrow$' and '$\downarrow$' with the precedence of operator $\downarrow$ being lower than that of the operator $\uparrow$. Operator $\uparrow$ is right associative while operator $\downarrow$, is left asociative.

$\uparrow$ is exponentiation operator, $a \uparrow b = a^b$

$\downarrow$ is logarithmic operator $a \downarrow b = \log_b a$

The value of the expression $(65536 \downarrow 2 \uparrow 4 \uparrow 2 \downarrow 2)$ is _____.

$\uparrow > \downarrow$

Right Gaut

$(65536 \downarrow 2 \uparrow 16 \downarrow 2$

$\log_{65536} 65536$

$a \uparrow b = a^b$

$a \downarrow b = \log_b a$

$\leftarrow 65536 \downarrow 65536 \downarrow 2$

$= 1 \downarrow 2$

$\log_2 1 = 0$

Slide



Topic : Question

Q Which of the following expressions evaluates to the largest number?

(A) The postfix expression 2 3 + 5 * 7 – ⟹ 55* ⇒ 257 - = 25-7 = 18

(B) The prefix expression + * – 2 3 5 7 ⇒ +2

(C) The infix expression (2 + 3) * (5 - 7) ⇒ 5 * –2 = -10

(D) The infix expression 2 + 3 * 5 – 7 ⇒ 2+ 15-7 = 10

+ * – 2 3 5 7

Infix

+ ab = a + b

+ * (2 - 3) 5 7

+ (2 - 3) * 5 , 7 = (2-3)*5+7 = -1 x 5 + 7 = 2

Slide

prefix

+*− 2,3,5,7

− * +

Right to left

a+b

+ a b

Tree

op operand$_1$

operand$_2$

top = operand$_1$

2							
3	2-3 = -1						
5		-1*5 = -5					
7		7	-5+7 = 2				



Topic : Question

#	postfix		Value
P	2 3 * 4 2 2 * / + 3 5 + -	A	80
Q	6 2 3 + - 3 8 2 / + * 2 $ 3 +	B	-1
R	5 2 * 3 4 + 5 2 * * +	C	52
S	8 2 3 ^ / 2 3 * + 5 1 * -	D	25
R	5 2 * 3 3 2 + * +	E	2
U	10 5 + 60 6 / * 8 –	F	142



Topic : Question

#Q.

Compute the post fix equivalent of the following expression $3 \times \log (x + 1) - a / 2$



2 mins Summary

Topic — brack in postfix

Topic — Infix to prefix

Topic — Evaluation of postfix, prefix

Topic — Stack life

Topic

t.me/Abhishekshaarma pw

Slide

THANK - YOU

Slide